5.1.7 Exercises for Section 5.1

Exercise 5.1.1: Design context-free grammars for the following languages:

* a) $\{a^i b^j c^k \mid i = j + k\}$

*! b) $\{a^i b^j \mid i \leq 2j\}$

! c) The set of all strings of 0's and 1s where the number of 0's is equal to the number of 1's.

!! d) The set of all strings of 0's and 1s where the number of 0's is not equal to the number of 1's.

Exercise 5.1.2: The following grammar generates the grammar of the language consisting of all strings of even length:

$$S \to AS \mid \varepsilon$$
$$A \to aa \mid ab \mid ba \mid bb$$

Give leftmost and rightmost derivations for the following strings:

* a) *aabbba*

b) *baabab*

c) *aaabbb*

! **Exercise 5.1.3:** Show that every regular language is a context-free language. *Hint*: Construct a CFG by induction on the number of operators in the regular expression.

! **Exercise 5.1.4:** A CFG is said to be *right-linear* if each production body has at most one variable, and that variable is at the right end. That is, all productions of a right-linear grammar are of the form $A \to wB$ or $A \to w$, where A and B are variables and w some string of zero or more terminals.

a) Show that every right-linear grammar generates a regular language. *Hint*: Construct an ϵ-NFA that simulates leftmost derivations, using its state to represent the lone variable in the current left-sentential form.

b) Show that every regular language has a right-linear grammar. *Hint*: Start with a DFA and let the variables of the grammar represent states.

*! **Exercise 5.1.5:** Let $T = \{0, 1, (,), +, *, \emptyset, e\}$. We may think of T as the set of symbols used by regular expressions over alphabet $\{0, 1\}$; the only difference is that we use e for symbol ϵ, to avoid potential confusion in what follows. Your task is to design a CFG with set of terminals T that generates exactly the regular expressions with alphabet $\{0, 1\}$.

Exercise 5.1.6: We defined the relation $\stackrel{*}{\Rightarrow}$ with a basis "$\alpha \Rightarrow \alpha$" and an induction that says "$\alpha \stackrel{*}{\Rightarrow} \beta$ and $\beta \Rightarrow \gamma$ imply $\alpha \stackrel{*}{\Rightarrow} \gamma$. There are several other ways to define $\stackrel{*}{\Rightarrow}$ that also have the effect of saying that "$\stackrel{*}{\Rightarrow}$ is zero or more $\Rightarrow$ steps." Prove that the following are true:

a) $\alpha \stackrel{*}{\Rightarrow} \beta$ if and only if there is a sequence of one or more strings

$$\gamma_1, \gamma_2, \ldots, \gamma_n$$

such that $\alpha = \gamma_1$, $\beta = \gamma_n$, and for $i = 1, 2, \ldots, n-1$ we have $\gamma_i \Rightarrow \gamma_{i+1}$.

b) If $\alpha \stackrel{*}{\Rightarrow} \beta$, and $\beta \stackrel{*}{\Rightarrow} \gamma$, then $\alpha \stackrel{*}{\Rightarrow} \gamma$. *Hint*: use induction on the number of steps in the derivation $\beta \stackrel{*}{\Rightarrow} \gamma$.

! **Exercise 5.1.7:** Consider the CFG G defined by productions:

$$S \to aS \mid Sb \mid a \mid b$$

a) Prove by induction on the string length that no string in $L(G)$ has ba as a substring.

b) Describe $L(G)$ informally. Justify your answer using part (a).

!! **Exercise 5.1.8:** Consider the CFG G defined by productions:

$$S \to aSbS \mid bSaS \mid \epsilon$$

Prove that $L(G)$ is the set of all strings with an equal number of a's and b's.

5.2 Parse Trees

There is a tree representation for derivations that has proved extremely useful. This tree shows us clearly how the symbols of a terminal string are grouped into substrings, each of which belongs to the language of one of the variables of the grammar. But perhaps more importantly, the tree, known as a "parse tree" when used in a compiler, is the data structure of choice to represent the source program. In a compiler, the tree structure of the source program facilitates the translation of the source program into executable code by allowing natural, recursive functions to perform this translation process.

In this section, we introduce the parse tree and show that the existence of parse trees is tied closely to the existence of derivations and recursive inferences. We shall later study the matter of ambiguity in grammars and languages, which is an important application of parse trees. Certain grammars allow a terminal string to have more than one parse tree. That situation makes the grammar unsuitable for a programming language, since the compiler could not tell the structure of certain source programs, and therefore could not with certainty deduce what the proper executable code for the program was.